

COMPANION
to
ChatGPT Teaches TikZ
Chapter 7: Trees (B)

Contents

7.1	What This Companion Adds	2
7.2	A Tree Vocabulary	3
7.3	Reading the Nesting	3
7.4	Spacing a Binary Tree	4
7.5	Choosing a Growth Direction	5
7.6	Styling Parent–Child Edges	6
7.7	Expression Trees	6
7.8	Worked Project: A Decision Tree	7
7.9	Debugging Workshop	9
7.10	Exercises	10
7.11	Solutions	10
7.12	ChatGPT Workshop	13
7.13	Final Checklist	13

7.1 What This Companion Adds

Chapter 7 introduced TikZ’s compact parent–child syntax. This Companion develops the habits needed to read nested source, control spacing, select a growth direction, and style every parent–child edge consistently.

A tree is different from a manually positioned network. Its hierarchy should be visible in the source: children are written inside the structure of their parent.

Design

Let the nesting express the hierarchy. Adjust level and sibling spacing before trying to repair individual nodes.

7.2 A Tree Vocabulary

Option or form	Purpose
<code>child {node {B}}</code>	Adds node B as a child.
<code>grow=down</code>	Uses the default downward direction.
<code>grow=right</code>	Makes levels progress to the right.
<code>grow=left</code>	Makes levels progress to the left.
<code>grow=up</code>	Makes levels progress upward.
<code>level distance</code>	Sets the distance between levels.
<code>sibling distance</code>	Sets the separation among children.
<code>edge from parent</code>	Styles every parent–child connection.

7.3 Reading the Nesting

A tree begins with a root node and ends with one semicolon.

```
\node {A}
  child {node {B}}
  child {node {C}};
```

Both B and C are children of A. Now compare:

```
\node {A}
  child {node {B}
    child {node {D}}
  }
  child {node {C}};
```

The braces after B contain D, so D is a child of B, not of A.

Common Mistake

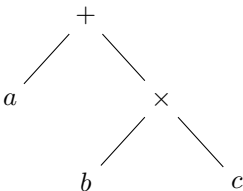
Do not put a semicolon after every child. One semicolon terminates the complete tree after all nested child structures have closed.

7.4 Spacing a Binary Tree

The two main controls solve different problems.

```
\begin{tikzpicture}[
  level distance=13mm,
  sibling distance=24mm
]
  \node {$+$}
    child {node {$a$}}
    child {node {$$\times$}}
      child {node {$b$}}
      child {node {$c$}}
  };
\end{tikzpicture}
```

See



Increase `level distance` when one generation crowds the next.
Increase `sibling distance` when nodes on the same level collide.

7.4.1 Spacing by Level

The top of a tree often needs more room than its lower branches. TikZ accepts options for individual levels:

```
\begin{tikzpicture}[  
  level 1/.style={sibling distance=30mm},  
  level 2/.style={sibling distance=16mm}  
]  
  ...  
\end{tikzpicture}
```

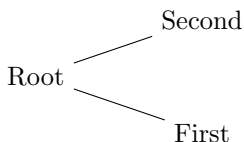
This preserves the tree model while giving each generation suitable spacing.

7.5 Choosing a Growth Direction

Trees grow downward by default. A right-growing tree can be better for a short hierarchy with long labels.

```
\begin{tikzpicture}[grow=right,level distance=24mm]  
  \node {Root}  
    child {node {First branch}}  
    child {node {Second branch}};  
\end{tikzpicture}
```

See



Choose the direction before fine-tuning distances. Changing orientation late can make earlier spacing adjustments irrelevant.

7.6 Styling Parent–Child Edges

Use one shared edge rule instead of repeating options on every child.

```
\begin{tikzpicture}[
  edge from parent/.style={draw,-{Stealth}}
]
  \node {Root}
    child {node {Left}}
    child {node {Right}};
\end{tikzpicture}
```

Node appearance can also be shared:

```
\begin{tikzpicture}[
  every node/.style={draw,rounded corners},
  edge from parent/.style={draw,-{Stealth}}
]
  ...
\end{tikzpicture}
```

The first style belongs to objects; the second belongs to relationships.

7.7 Expression Trees

The source of an expression tree should mirror the expression's grouping. For $(a + b)c$, multiplication is the root and addition is its left child.

```
\begin{tikzpicture}[
  level distance=13mm,
  sibling distance=24mm
]
  \node {\times}
    child {node {+}}
    child {node {a}}
    child {node {b}}
  }
  child {node {c}};
\end{tikzpicture}
```

Parentheses in the mathematical expression determine nesting in the TikZ source.

7.8 Worked Project: A Decision Tree

The project separates the hierarchy, spacing, and visual style.

7.8.1 Stage 1: Write the Hierarchy

```
\node {Problem}
  child {node {Known}
    child {node {Apply rule}}
  }
  child {node {Unknown}
    child {node {Find data}}
  };
```

7.8.2 Stage 2: Add Spacing

```
level distance=15mm,
sibling distance=36mm
```

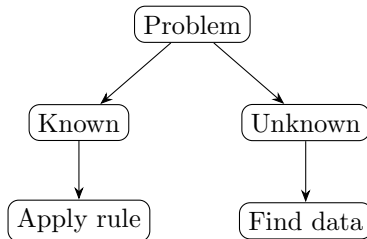
7.8.3 Stage 3: Add Shared Styles

```
every node/.style={draw,rounded corners},
edge from parent/.style={draw,-{Stealth}}
```

7.8.4 The Complete Picture

```
\begin{tikzpicture}[
  level distance=15mm,
  sibling distance=34mm,
  every node/.style={draw,rounded corners},
  edge from parent/.style={draw,-{Stealth}}
]
\node {Problem}
  child {node {Known}
    child {node {Apply rule}}}
  child {node {Unknown}
    child {node {Find data}}}
  ;
\end{tikzpicture}
```

See



Design

The nested source states the hierarchy. The picture options control its appearance. Either part can be revised without reconstructing the other.

7.9 Debugging Workshop

7.9.1 Forgetting the Final Semicolon

The complete root command, including all children, ends with a semicolon:

```
\node {A}  
  child {node {B}}  
  child {node {C}};
```

7.9.2 Unbalanced Braces

Indent every child beneath its parent. Matching indentation makes a missing closing brace much easier to find.

7.9.3 Changing the Wrong Distance

If generations collide, change `level distance`. If siblings collide, change `sibling distance`. Do not change both before identifying the direction of the crowding.

7.9.4 Moving Individual Nodes

Manual coordinates hide the hierarchy. First change the growth direction, global spacing, or a level-specific spacing rule.

7.10 Exercises

1. Draw a root with three children.
2. Give the first child two children of its own.
3. Draw a binary tree of depth two.
4. Increase only the distance between levels.
5. Redraw a downward tree with **grow=right**.
6. Give every node the same border and every edge the same arrow tip.
7. Draw an expression tree for $(a + b)(c + d)$.
8. Extend the worked project by giving each final action two outcomes.

7.11 Solutions

7.11.1 Exercises 1–4

```
\node {A}
  child {node {B}}
  child {node {C}}
  child {node {D}};
```

```
\node {A}
  child {node {B}
    child {node {D}}
    child {node {E}}
  }
  child {node {C}};
```

```

\begin{tikzpicture}[
  level distance=16mm,
  sibling distance=24mm
]
  \node {A}
    child {node {B}
      child {node {D}}
      child {node {E}}
    }
    child {node {C}
      child {node {F}}
      child {node {G}}
    };
\end{tikzpicture}

```

Exercise 4 changes only the value assigned to `level distance`.

7.11.2 Exercises 5–7

```

\begin{tikzpicture}[grow=right]
  \node {A}
    child {node {B}}
    child {node {C}};
\end{tikzpicture}

```

```

\begin{tikzpicture}[
  every node/.style={draw},
  edge from parent/.style={draw,-{Stealth}}
]
  ...
\end{tikzpicture}

```

```

\node {$\times$}
  child {node {$+$}
    child {node {$a$}}
    child {node {$b$}}
  }
  child {node {$+$}
    child {node {$c$}}
    child {node {$d$}}
  };

```

For Exercise 8, add two nested `child` structures inside each action node; keep the shared styles unchanged.

7.12 ChatGPT Workshop

Ask ChatGPT to preserve the hierarchy unless the prompt explicitly requests a different tree.

1. Explain this tree's nesting and identify the parent of every node.
2. Improve this tree using **level distance** and **sibling distance** only.
3. Convert the expression $(a + b)(c + d)$ into a TikZ expression tree.
4. Find the unbalanced brace in this indented tree without redesigning it.
5. Compare downward and right-growing versions of this hierarchy and recommend the clearer orientation for a narrow page.

7.13 Final Checklist

Before continuing to Chapter 8, check that you can

1. read parent–child relationships from nested braces;
2. end the complete tree with one semicolon;
3. distinguish level spacing from sibling spacing;
4. choose a suitable growth direction;
5. style all parent–child edges consistently;
6. translate the grouping of an expression into a tree; and
7. extend a hierarchy without manually placing every node.